

boaz_학기 세션 과제(1)

GPT

1. Abstract

2. Introduction

2.1. 기존 모델의 문제점

2.2. 2-stage 학습 제안

2.3. 기술적 분석

2.4. Related works

3. Method

3.1. Unsupervised pre-training

3.2. Supervised fine-tuning

3.3. Task-specific input transformations

3.3.1. Classification(분류)

3.3.2. Textual entailment(자연어 추론)

3.3.3. Similarity(문장 유사도)

3.3.4. Question Answering and Commonsense Reasoning(질의응답 & 상식추론)

4. Experiment

4.1. Setup

4.1.1. unsupervised pre-training

4.1.2. supervised fine-tuning

4.2. Results

4.3. Analysis

5. Conclusion

6. Limitations

Deepseek-R1

1. Abstract

2. Introduction

2.1. Reasoning

2.2. CoT (Chain-of-Thought) 프롬프팅

2.3. 강화학습(RL) 기반 접근법 제안

2.4. Contributions

3. Method

Overview

3.1 Deepseek-R1-zero

3.1.1. 강화학습

3.1.2. PPO

3.1.3. GRPO

- [3.1.4. 2가지 규칙 기반 보상 시스템](#)
- [3.1.5. 훈련 템플릿\(Training Template\)](#)
- [3.1.6. AHA moment](#)
- [3.1.7. 한계점](#)
- [3.2. DeepSeek-R1](#)
 - [3.2.1 Cold Start SFT](#)
 - [3.2.2. First RL Stage](#)
 - [3.2.3. Rejection Sampling & SFT](#)
 - [3.2.4. Second RL Stage](#)
- [4. Experiment](#)
- [5. Conclusion](#)
- [6. Limitations](#)

GPT

1. Abstract

- **NLP는 다음의 다양한 작업 포함**
 - 텍스트 함의(textual entailment)
 - 질문 응답(question answering)
 - 의미적 유사도 평가(semantic similarity assessment)
 - 문서 분류(document classification)
- **이러한 특정 작업을 학습하기 위한 라벨링된 데이터는 매우 부족한 상태**
 - 판별적 학습(discriminative training)으로 학습된 모델이 충분한 성능을 내기 어려움
- **대규모의 비라벨링 데이터(unlabeled text corpus)는 많이 존재**
 - 이를 활용한 unsupervised pre-training와 supervised fine-tuning 수행 제안

2. Introduction

2.1. 기존 모델의 문제점

- 지도학습 의존성이 너무 큼

- 기존 딥러닝 방식은 Labeled Data가 매우 많이 필요함
- 비싸고 도메인 확장에 취약
- **목적 함수의 불확실성**
 - 어떤 방식으로 학습시켜야 나중에 다른 작업에 적합한지 불분명
- **target task에 대한 전이 방식의 부재**
 - 학습된 지식을 target task로 어떻게 효과적으로 전이할지에 대한 표준이 없음

2.2. 2-stage 학습 제안

- **1단계 (unsupervised pre-training)**
 - 라벨 없는 대규모 텍스트로 language modeling objective 학습
 - 언어의 일반적 지식(문법, 의미, 상식적 패턴)을 파라미터에 저장
- **2단계 (supervised fine-tuning)**
 - 학습된 파라미터를 target task에 맞춰 미세 조정
 - task 맞춤 헤드/입력 포맷 정도의 최소 변경만 허용

2.3. 기술적 분석

- attention 메커니즘으로 텍스트 속 장기 의존성을 잘 포착해 긴 문맥을 잘 다루고 대규모 학습이 가능
- 구조 변경 없이 fine-tuning이 쉬움

2.4. Related works

- **Semi-supervised learning for NLP(준지도 학습)**
 - 라벨 데이터 + 비라벨 데이터
 - 초기 - 라벨 없는 대규모 텍스트 -> 단어/구 빈도 등의 통계치 계산
 - 지도 학습 모델의 입력 특징으로 활용
 - 중기 - word embeddings(word2vec) 등장
 - 라벨 없는 대규모 텍스트로 단어의 의미를 학습해 성능을 높임
 - 단어 수준(word level)의 정보 전달에 그침
- **Unsupervised pre-training(비지도 학습)**
 - 라벨 없이 입력 데이터 만으로 학습

- pre-training이 딥러닝 모델의 일반화를 돕는 역할을 하는 것이 입증됨
- transformer - self-attention, 거리 간 의존성 문제 완화, 병렬 학습 가능 => 더 강한 전이 성능을 기대
- **Auxiliary training objectives(학습을 돕기 위한 보조 목적함수)**
 - 메인 task를 학습하면서 다른 task를 같이 학습
 - 초기 - 여러 NLP task를 한 모델에서 함께 학습해 의미적 표현을 더 잘 만들고자함
 - 현재 - 이 함수를 사용하긴 하지만 unsupervised pre-training으로 언어적 특징을 충분히 학습하는것을 강조

3. Method

3.1. Unsupervised pre-training

$$L_1(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, \dots, u_{i-1}; \Theta)$$

- 앞 문맥을 보고 다음 단어 확률의 최댓값을 구하기
 - k - context window 사이즈(문맥 길이)
 - Θ - 뉴럴 네트워크의 파라미터
 - u_i - i번째 token
 - $u_{i-k}.. u_{i-1}$ - 현재 단어 u_i 를 예측할 때 참고하는 이전 문맥

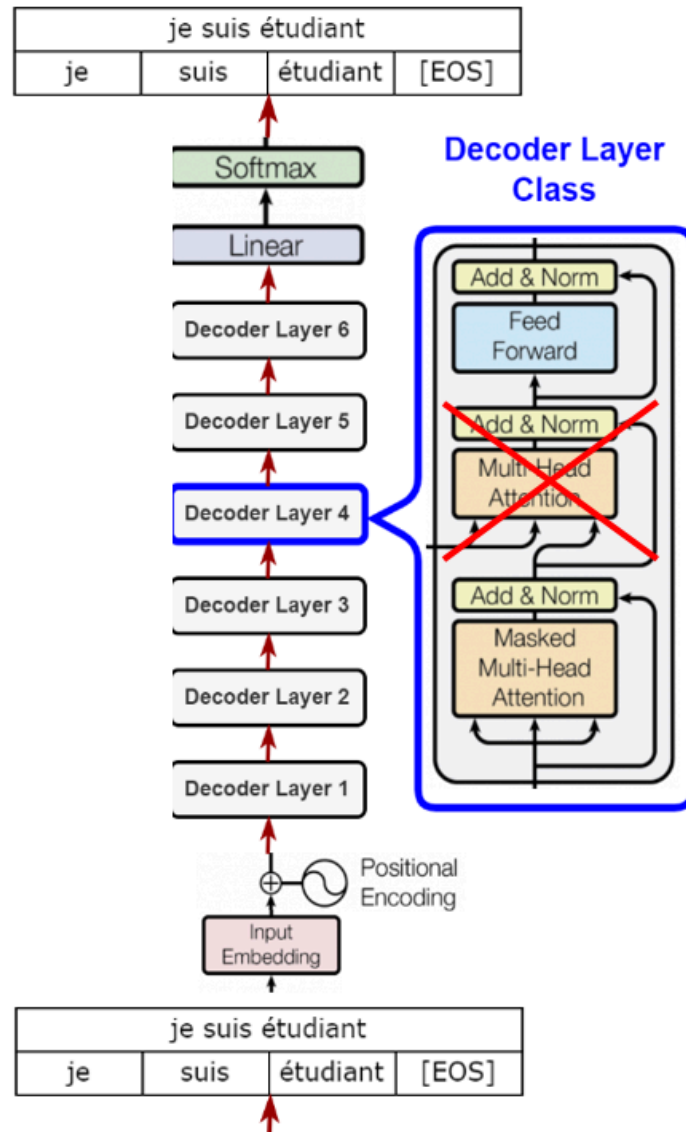
⇒ $u_{i-k}.. u_{i-1}$ 이 주어졌을 때 u_i 의 확률값을 계산

- 입력 → 연산 → 출력의 방식

$$\begin{aligned} h_0 &= UW_e + W_p \\ h_l &= \text{transformer_block}(h_{l-1}) \forall l \in [1, n] \\ P(u) &= \text{softmax}(h_n W_e^T) \end{aligned}$$

- 입력 임베딩

- 각 토큰의 초기 벡터는 단어 의미+위치 정보
- transformer block 통과



- 같은 구조의 트랜스포머 블록을 n번 쌓아 정보를 추상화함
 - Transformer의 decoder부분만 사용함
 - token 분포 출력
 - hn - 마지막 레이어의 hidden state
 - WeT - 입력 임베딩 행렬의 전치
 - 전치 하는 이유
1. 행렬 연산의 차원을 맞추기 위해

⇒ 다음 단어가 나올 확률의 결과값 벡터 = $h_n * W_e$ 에 전치한 벡터

2. 가중치 공유를 위해

⇒ 별도의 행렬을 만들기보다 파라미터 절약과 학습 효율성을 위해 가중치를 공유하도록 행렬 전치

3.2. Supervised fine-tuning

$$P(y|x^1, \dots, x^m) = \text{softmax}(h_l^m W_y).$$

- pre-training으로 언어에 대한 이해가 생긴 모델로 실제 target task를 풀게 함
- supervised learning 이므로 task에 따라 시퀀스에 따른 label값이 존재
- 입력 문장($x^1.. x^m$)을 pre-trained된 transformer 모델에 통과시켜 활성화 값 얻음
 - Position-wise layer와 softmax layer 사이에 linear layer 하나를 추가하여 각 task마다 label y 를 예측

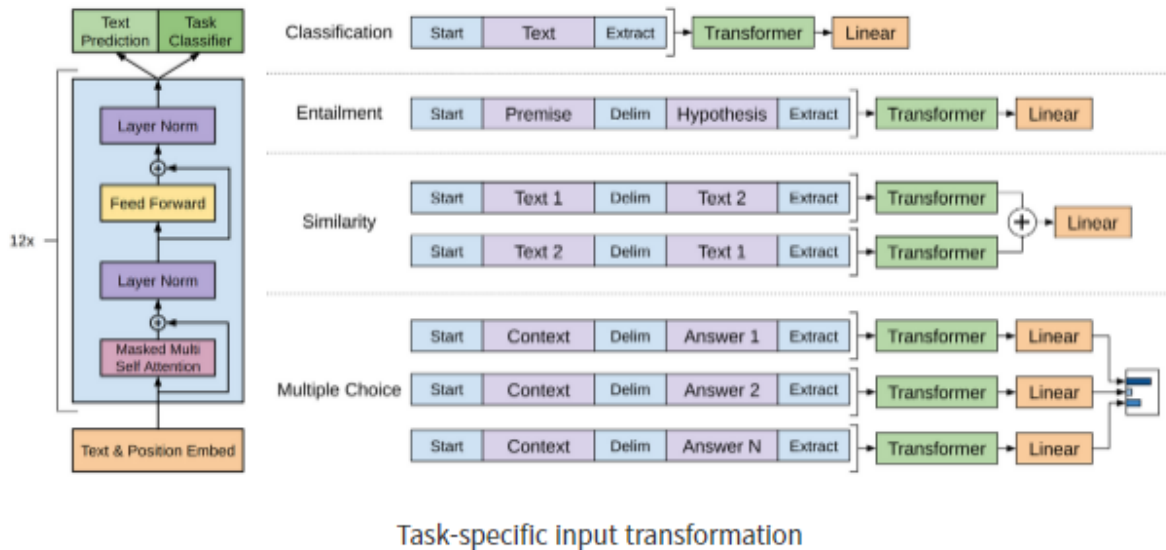
$$L_2(\mathcal{C}) = \sum_{(x,y)} \log P(y|x^1, \dots, x^m).$$

- 최대화 해야할 목적 함수

$$L_3(\mathcal{C}) = L_2(\mathcal{C}) + \lambda * L_1(\mathcal{C})$$

- fine tuning 할 때 unsupervised learning도 가중치를 곱해 pre-training loss를 보조 목적 함수로 더하여 최종 fine-tuning loss 계산

3.3. Task-specific input transformations



- GPT-1 모델은 task전용 레이어를 쌓는것이 아닌 구조는 고정하고 입력 데이터의 배치만 바꿈
- 학습을 최소한의 변화로 가능하게 함
- 주요 토큰
 - <s> - 문장 시작
 - <e> - 문장 끝
 - <\$> - 문장과 문장 사이를 구분하는 구분자

3.3.1. Classification(분류)

- 기존 분류 문제

3.3.2. Textual entailment(자연어 추론)

- 전제(premise)와 가정(hypothesis) 시퀀스 사이에 <\$>을 추가해 하나의 시퀀스로 만듦

3.3.3. Similarity(문장 유사도)

- 두문장(A,B)의 유사성을 비교
- 전제와 가정처럼 순서가 존재하지 않아 문장 순서를 반영하기 위해 (A,B), (B,A)모두 각각 모델에 forward진행
- 마지막 linear layer에 들어가기전 element-wise로 합하여 출력

3.3.4. Question Answering and Commonsense Reasoning(질의 응답 & 상식추론)

- task에 대해 지문 z , 질문 q , 정답 리스트($a_1, a_2 \dots a_k$)가 존재
- 각 후보마다 하나의 시퀀스 생성 후 그 후보들 사이에서 softmax 연산으로 가장 확률 높은 답 선택

4. Experiment

4.1. Setup

4.1.1. unsupervised pre-training

- dataset
 - unsupervised pre-training에 BooksCorpus 데이터셋을 사용
 - 7,000개가 넘는 다양한 장르의 출간되지 않는 책들로 구성
 - 책 데이터라 long term dependency 학습 가능
- model specification
 - transformer의 decoder만 활용 (총 12 layer)

4.1.2. supervised fine-tuning

- 특별한 task specific 설계없이 단일 language model 활용
- pre-training에서 사용한 하이퍼파라미터를 대부분 유지함
- 활성화 함수 ReLU대신 GeLU를 활용해 학습 효율을 높임

4.2. Results

- 자연어 추론
 - 크기가 작은 dataset보다 큰 경우에서 더 좋은 성능을 보임

Method	MNLI-m	MNLI-mm	SNLI	SciTail	QNLI	RTE
ESIM + ELMo [44] (5x)	-	-	<u>89.3</u>	-	-	-
CAFE [58] (5x)	80.2	79.0	<u>89.3</u>	-	-	-
Stochastic Answer Network [35] (3x)	<u>80.6</u>	<u>80.1</u>	-	-	-	-
CAFE [58]	78.7	77.9	88.5	<u>83.3</u>		
GenSen [64]	71.4	71.3	-	-	82.3	59.2
Multi-task BiLSTM + Attn [64]	72.2	72.1	-	-	82.1	61.7
Finetuned Transformer LM (ours)	82.1	81.4	89.9	88.3	88.1	56.0

- 질의응답
 - story cloze test, RACE dataset 모두 좋은 성능을 보임 -> 모델이 긴 맥락을 잘 다룬다

Method	Story Cloze	RACE-m	RACE-h	RACE
val-LS-skip [55]	76.5	-	-	-
Hidden Coherence Model [7]	<u>77.6</u>	-	-	-
Dynamic Fusion Net [67] (9x)	-	55.6	49.4	51.2
BiAttention MRU [59] (9x)	-	<u>60.2</u>	<u>50.3</u>	<u>53.3</u>
Finetuned Transformer LM (ours)	86.5	62.9	57.4	59.0

- 문장 유사도 + 분류
 - CoLA dataset, SST dataset SOTA 달성

Method	Classification		Semantic Similarity			GLUE
	CoLA (mc)	SST2 (acc)	MRPC (F1)	STSB (pc)	QQP (F1)	
Sparse byte mLSTM [16]	-	93.2	-	-	-	-
TF-KLD [23]	-	-	86.0	-	-	-
ECNU (mixed ensemble) [60]	-	-	-	<u>81.0</u>	-	-
Single-task BiLSTM + ELMo + Attn [64]	<u>35.0</u>	90.2	80.2	55.5	<u>66.1</u>	64.8
Multi-task BiLSTM + ELMo + Attn [64]	18.9	91.6	83.5	72.8	<u>63.3</u>	<u>68.9</u>
Finetuned Transformer LM (ours)	45.4	91.3	82.3	82.0	70.3	72.8

4.3. Analysis

- impact of number of layers transferred
 - layer수 증가량에 따른 RACE와 MutliNLI에 대한 실험 진행
 - layer수가 증가할 수록 성능이 꾸준히 향상

⇒ 사전 학습 모델의 각 layer가 target task를 해결하는데 필요한 여러 언어적 기능을 분담해서 학습(pretraining이 유용하는것을 증명)

- zero-shot behaviors
 - pre-train 진행만으로 downstream task에 대한 성능을 보여주는 실험
 - pretraining이 진행될수록 zero-shot성능이 꾸준히 상승하고 LSTM보다높은 성능을 보여줌
- ablation studies
 - LM 목적함수 제거
 - 대규모 데이터셋에서 성능 하락, 데이터가 클수록 효과가 큼
 - 모델 교체(transformer ⇒ LSTM)
 - 평균점수 감소
 - pretraining 제거
 - 성능 14.8% 감소

5. Conclusion

- task별로 architecture를 설계해야 하는 것이 아닌 2-stage 학습 제안
- 특히 unsupervised pre-training이 downstream task 성능 향상에 크게 기여
- 다양한 긴 문장을 pre-train 함으로써 여러 task에서 SOTA 달성
- 하나의 pretrained LM을 다양한 nlp task에 활용 가능

6. Limitations

- 모델 크기가 작음
- 데이터 규모가 제한적 (bookscorpus only)
- 모든 task에 대해 fine-tuning 필요

Deepseek-R1

1. Abstract

- 기존 모델의 reasoning 문제
 - 프롬프팅 기법의 발전으로 reasoning task에서 상당 성과를 이룸
 - 그러나 여전히 인간이 작성한 추론 data에 의존하며 복잡한 task에 대해 모델 능력이 부족함
 - 인간이 라벨링한 data가 아닌 RL만으로 언어 모델의 reasoning 능력을 향상시키는 제안
- DeepSeek-R1-Zero (이전 모델)
 - 강화학습(RL)을 이용해 훈련
 - 지도학습(SFT) 없이도 뛰어난 추론 능력
 - 가독성 저하와 언어 혼합 문제 등의 한계점 발생
- DeepSeek-R1 (논문에서 제안한 모델)
 - multi-stage training, cold-start data를 RL 이전 단계에 활용
 - OpenAI-o1-1217을 약간 상회하거나 비슷
 - 전문가 수준의 코딩 능력

2. Introduction

2.1. Reasoning

- 추론능력은 인간 지능의 핵심 능력이며 다양하고 복잡한 문제 해결에 사용
- 인공지능 시스템 또한 추론 능력이 복잡한 문제를 해결하는데 필수적인 핵심 요소

2.2. CoT (Chain-of-Thought) 프롬프팅



You

CoT 방식으로 답을 해주세요



ChatGPT

숫자 4, 8, 9, 15, 12, 2, 1 중에서 홀수를 찾습니다. 이 숫자들 중 홀수는 9, 15, 1입니다. 이제 이 홀수들의 합을 계산해야 합니다.

$9 + 15 + 1 = 25$

따라서, 이 숫자들의 홀수 합은 25입니다.

- 모델이 **단계별** 추론 과정을 생성하도록 유도해 복잡한 문제 해결 능력을 향상시킴
- 이러한 접근 방식에는 한계점이 존재
 - 인간이 만든 대규모 데이터에 의존함
 - 데이터 생성 비용이 매우 높음
 - 다양한 문제 유형을 충분히 포괄하기 어려움
 - 모델의 잠재적인 능력을 제한할 수 있음

2.3. 강화학습(RL) 기반 접근법 제안

- 대규모 언어 모델이 인간이 작성한 reasoning data 없이 스스로 추론 능력을 학습할 수 있는가
- 핵심 아이디어
 - 정답 여부가 검증 가능한 수학, 코딩 문제를 활용함
 - 모델이 생성한 답변을 평가해 reward 제공
 - 강화학습으로 모델이 더 나은 추론 전략을 학습하도록 함
- 결과
 - 문제 해결 중 스스로 자기 검증
 - 중간 결과 재확인, 반성(reflection)
 - 긴 CoT 생성

⇒ 모델이 단순히 정답을 생성하는 것이 아닌 추론 능력을 학습함을 시사

2.4. Contributions

- 순수 강화학습(RL) 적용
 - SFT 없이 순수 RL만으로 LLM의 추론 능력을 향상시킬 수 있음을 입증한 최초의 연구
- DeepSeek-R1 개발 파이프라인
 - 2-stage의 RL과 SFT를 포함한 파이프라인
 - RL 단계는 향상된 추론 패턴을 발견하고 인간의 선호를 맞추는 데 중점
 - SFT 단계는 모델 추론, 비추론 능력의 seed 역할

3. Method

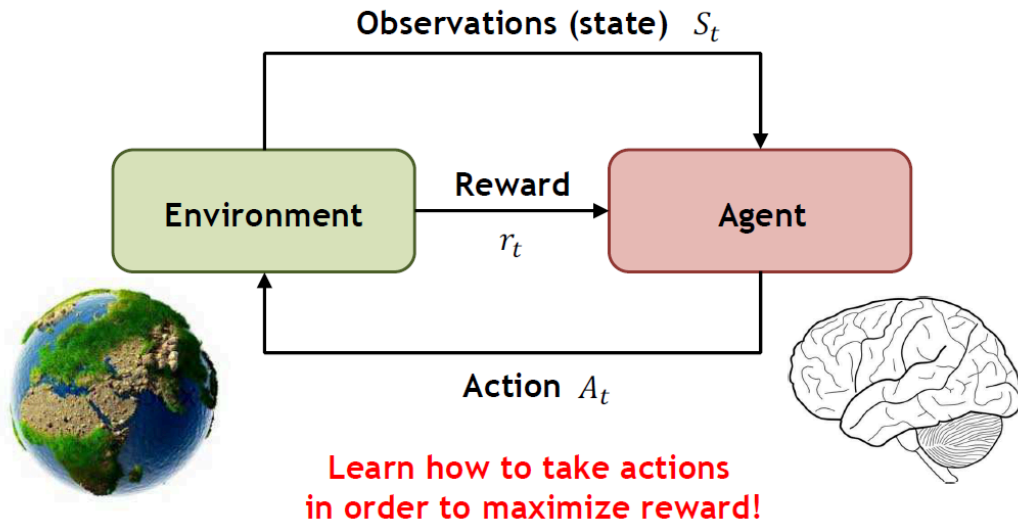
Overview

1. 모델을 policy로 간주
 2. 모델이 생성한 추론 과정+답변 평가
 3. 정답 여부를 기반으로 reward 제공
 4. RL 알고리즘으로 모델 업데이트
- Deepseek-R1-zero
 - RL 기반 모델을 직접 적용한 사례
 - Deepseek-R1
 - 긴 CoT 데이터를 활용해 미세 조정된 체크포인트에서 RL을 시작한 모델

3.1 Deepseek-R1-zero

3.1.1. 강화학습

- Reinforcement Learning Algorithm (강화학습 알고리즘)



prompt → 모델 응답 → reward

- Policy
 - 어떤 상태에서 어떤 행동을 할 확률

$$\pi(a \mid s)$$

- Policy Gradient
 - reward 기대값을 최대화하기 위한 방향으로 파라미터를 업데이트함

$$\nabla_{\theta} J(\theta) = E[R \nabla_{\theta} \log \pi_{\theta}(a \mid s)]$$

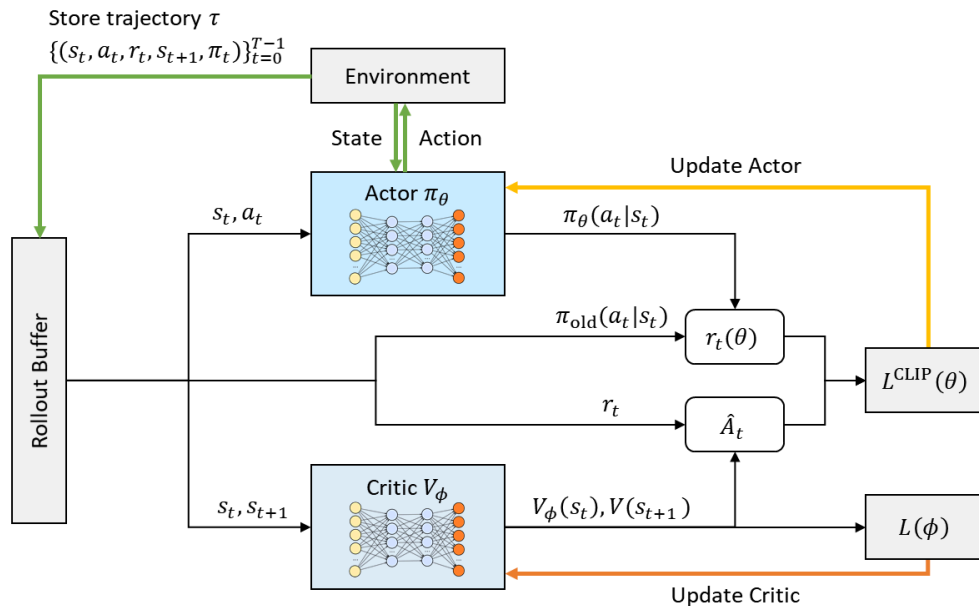
- 그러나 high variance로 인해 학습 속도가 느리고 불안정함
- Advantage
 - reward 대신 advantage 사용 (평균 reward보다 좋으면 +, 아니면 -)
- Actor-Critic
 - Actor (policy 모델) - 행동 선택

$$\pi_{\theta}(a \mid s)$$

- Critic (value 모델) - baseline 계산

$$V(s)$$

3.1.2. PPO



- 기존의 policy gradient 문제를 해결하기 위해 policy 업데이트 크기 제한

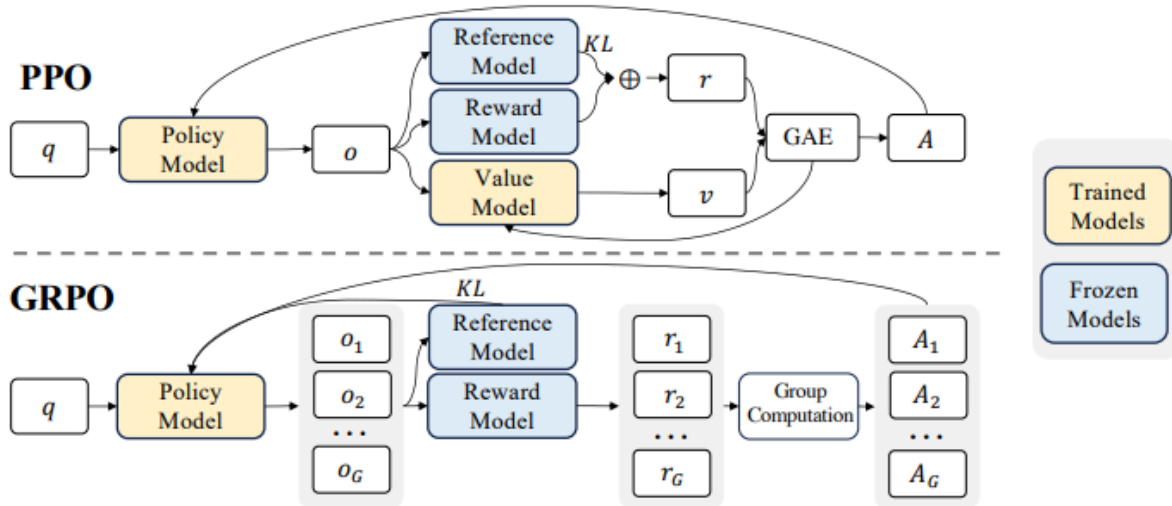
$$r(\theta) = \pi_{old}(a | s) / \pi_\theta(a | s)$$

- 기존 policy 대비 새로운 policy의 확률비가 일정 범위를 넘지 않도록 클리핑된 objective를 최대화함
- policy 업데이트 폭을 제한해 학습 폭주를 막고 데이터 효율성과 안정성을 확보

• PPO 문제점

- Actor-Critic 구조인 PPO는 LLM 크기 + critic 파라미터로 인해 메모리를 많이 사용
- Value prediction 어려움
- 긴 reasoning에서 불안정함

3.1.3. GRPO



- 강화학습의 비용을 절감하기 위해 도입한 기법
- policy 최적화를 안정적이고 효율적으로 수행할 수 있음
- policy 모델과 같은 크기의 critic 모델을 사용하는 대신 그룹 점수를 통해 baseline 추정
- GRPO 작동 방식

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \epsilon, 1 + \epsilon \right) A_i \right) - \beta \mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) \right), \quad (1)$$

- 각 질문 q에 대해 old policy에서 G개의 output sample을 뽑아냄
- 목적함수는 policy 비율을 $1-\epsilon, 1+\epsilon$ 사이로 제한하는 **클리핑 기법**을 포함하고 **KL 발산 패널티**를 통해 안정성을 유지함
- 즉 목적함수를 최대화하는 방식으로 **policy model**을 최적화하는 것
- GRPO 수식 뜯어보기

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \epsilon, 1 + \epsilon \right) A_i \right) - \beta \mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) \right), \quad (1)$$

- $E[\cdot]$: 기댓값(Expectation)
- clip : old policy 확률값과 new policy 확률값의 비율을 $1-\epsilon, 1+\epsilon$ 사이로 제한
- Advantage 함수 :

$$\min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \epsilon, 1 + \epsilon \right) A_i \right)$$

- A_i : i번째 정책에 주어지는 advantage (상대적인 보상)

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}. \quad (3)$$

⇒ 원 보상 (r_i)로부터 보상값들 집합 평균을 빼고 보상 표준편차로 나누어 상대적 인 보상으로 정규화함

⇒ A_i 가 음수면 해당 행동이 상대적으로 보상이 낮아 새로운 policy에서는 그 행동 확률을 줄이는 방향으로 업데이트 할것을 싣

- min()로 비율* A_i (클리핑x)와 clip한 비율* A_i (클리핑o)중 더 **작은 값**을 고름
⇒ clip만 쓴다면 너무 과도하게 업데이트를 제한할 수 있으므로 **더 작은 값을 목적함수로 골라 안정적인 학습을 유도함**

- KL 발산 패널티 :

$$\mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) = \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - \log \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - 1, \quad (2)$$

- 두 확률분포 간 차이를 나타내는 척도로 GRPO에서는 패널티의 개념으로 활용
- $\pi_{\theta_{old}}(o | q)$ - 이전 단계에서 학습된 정책 모델(Old Policy)의 확률 분포를 의미
- $\pi_{ref}(o | q)$ - 참조 모델(Reference Policy)의 확률 분포를 의미

• 수식 해석

- GRPO는 min값에 KL 발산 패널티에 하이퍼파라미터를 곱해 빼줌

- 두 policy model 차이가 클수록(KL 발산값이 클수록) 감점이 많음
- 결과가 최대화되기 위해서는 KL 발산값이 작은 쪽으로 최적화가 진행된 것
 - 클리핑과 상대적 보상, 패널티로 학습의 안정성을 높임

3.1.4. 2가지 규칙 기반 보상 시스템

- 학습 시 신경망 기반 reward model을 적용하지 않아 보상 해킹에 취약해 보상 시스템을 채택해 학습 효율성을 극대화
1. 정확도 보상(Accuracy Rewards)
 - 모델 응답이 올바른지 평가
 - 결과를 명확하게 판별 가능한 문제에 대해 신뢰할 수 있는 검증 시스템을 적용
 - 수학 문제 : 지정된 형식으로 응답하도록 해 신뢰성 있는 규칙 기반 평가가 가능
 - LeetCode 문제 : 컴파일러를 사용해 사전에 정의된 test case를 기반으로 모델 응답 평가
 2. 형식 보상(Format Rewards)
 - 모델이 사고 과정을 <think>와 </think>태그 사이에 작성하도록 강제하는 보상 모델을 활용
 - 체계적인 사고과정을 명시하도록 유도해 논리적인 추론 구조를 형성

3.1.5. 훈련 템플릿(Training Template)

- 구조적 요구사항
 - 모델이 먼저 추론 과정을 생성한 뒤 최종 답변을 출력해야함
 - 템플릿은 이와 같은 형식을 따르도록 모델에게 요구함
- 내용에 대한 제약 최소화
 - 반사적 추론(reflective reasoning)을 강제하거나 특정 문제 해결 전략을 유도하는 등 내용에 대한 특정 편향(content-specific biases)은 의도적으로 배제
 - 모델이 다양한 방식으로 문제를 해결하도록 유연성 보장

3.1.6. AHA moment

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to

Response: <think>

To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both ...

$$(\sqrt{a - \sqrt{a + x}})^2 = x^2 \implies a - \sqrt{a + x} = x^2.$$

Rearrange to isolate the inner square root term:

$$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$$

...

Wait, wait. Wait. That's an **aha moment** I can flag here.

Let's reevaluate this step-by-step to identify if the correct sum can be ...

We started with the equation:

$$\sqrt{a - \sqrt{a + x}} = x$$

First, let's square both sides:

$$a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$$

Next, I could square both sides again, treating the equation: ...

...

- 훈련 중 모델의 self-reflection으로 자신의 추론 과정을 스스로 다시 검토하는 현상
- 명시적으로 가르치지 않아도 강화학습을 통해 self-verification과 reflective reasoning을 스스로 학습했음을 보여줌

⇒ 강화학습만으로도 reasoning 능력이 자연스럽게 발전할 수 있음을 의미

3.1.7. 한계점

- 읽기 어려운 출력과 언어 품질이 나빠 사람이 이해하기 어려움

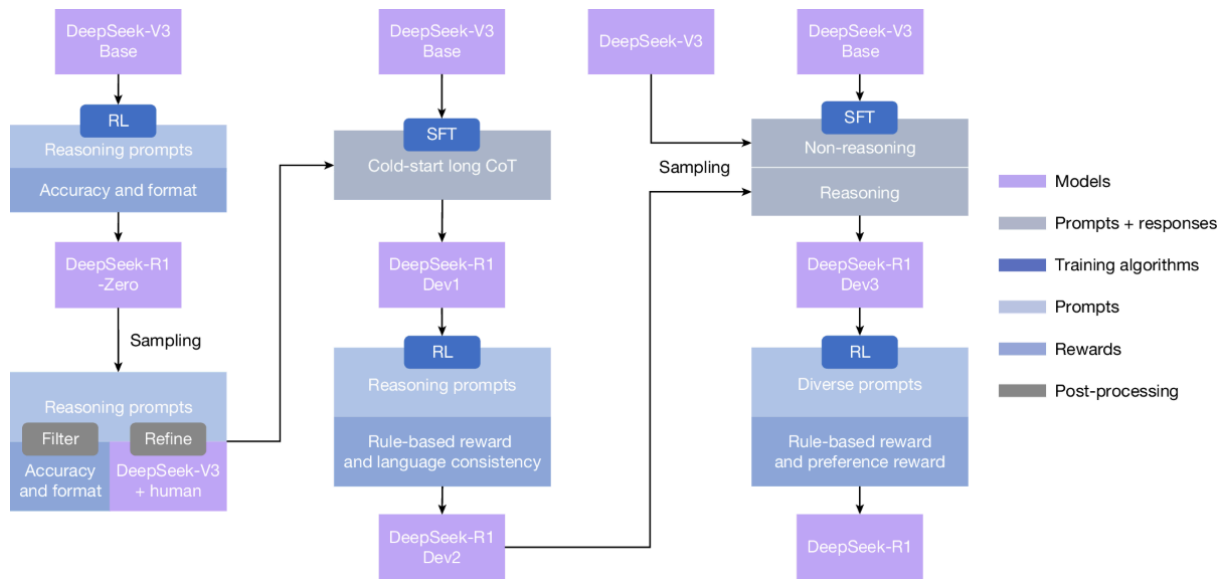
```
hmm maybe wrong
try again
calc 32+19=51 wait
maybe mistake
```

- 언어 혼합 (하나의 응답 안에서 영어와 중국어가 섞여 나타나는 현상)

```
English + Chinese + code
```

- 불안정한 reasoning

3.2. DeepSeek-R1



3.2.1 Cold Start SFT

- 초기 강화학습 단계에 발생하는 불안정을 방지하기 위해 소량의 CoT 데이터를 수집 및 생성
- 모델을 미세조정하고 이를 통해 강화학습의 초기 모델로 활용함
- 데이터 수집 방법
 - 긴 CoT 예제를 활용한 few-shot prompting 진행
 - reflection과 검증을 포함한 자세한 답변을 생성하도록 직접 프롬프트
 - 읽기 좋은 형식으로 변환
 - Human annotators가 후처리하여 결과 정제

3.2.2. First RL Stage

- 추론 성능+언어 혼합 문제 개선
- 언어 일관성을 보장하는 reward를 도입해 모델이 추론 작업에서 convergence에 도달할때까지 RL 훈련 진행
 - 규칙 기반 Reward
 - Language Consistency Reward
 - CoT에서 목표 언어 단어의 비율로 계산
- 사용자가 읽기 편하도록 가독성 확보 및 언어 혼용 문제 개선

3.2.3. Rejection Sampling & SFT

- 강화학습 결과를 정제해 다시 학습
- Reasoning Data
 - RL 체크포인트에서 rejection sampling을 통해 추론 프롬프트와 추론 경로 데이터 생성
- Non-Reasoning Data
 - 작성, 사실 기반 QA, self-cognition, 번역 등
 - 특정 작업에 대해 DeepSeek-V3의 SFT 데이터셋을 활용해 CoT를 생성 후 질문에 답변
 - 단순 질의에 대해서는 CoT 없이 직접 응답

3.2.4. Second RL Stage

- 인간 선호도를 반영하기 위해 보상 모델을 활용
- Model-based Rewards
 1. 유익성 보상(Helpful Reward Model)
 - ⇒ 응답이 사용자에게 실제로 도움이 되는지 평가
 - 평가 기준1. 사용자 질문에 얼마나 관련성이 있는가?
 - 평가 기준2. 실제로 도움 되는 정보를 제공하는가?

$$Reward_{helpful} = RM_{helpful}(ResponseA, ResponseB)$$

2. 안정성 보상(Safety Reward Model)

⇒ 모델이 생성하는 내용이 안전하고 책임 있는 방식으로 제공되는지 평가

- 평가 기준1. 잠재적 위험 및 유해 콘텐츠 식별
- 평가 기준2. 사회적 편향성 제거
- 평가 기준3. 정해진 안전 가이드라인 준수 여부

$$Reward_{safety} = RM_{safety}(Response)$$

- 데이터별 보상 전략
 - 추론 데이터 : 정답 여부로 평가하는 rule-based reward

- 일반 데이터 : 사람이 좋아하는지 평가하는 reward model

⇒ 추론 성능 유지 + 인간 선호도 반영

4. Experiment

	Benchmark (Metric)	R1-Zero	R1-Dev1	R1-Dev2	R1-Dev3	R1
English	MMLU (EM)	88.8	89.1	91.2	91.0	90.8
	MMLU-Redux (EM)	85.6	90.0	93.0	93.1	92.9
	MMLU-Pro (EM)	68.9	74.1	83.8	83.1	84.0
	DROP (3-shot F1)	89.1	89.8	91.1	88.7	92.2
	IF-Eval (Prompt Strict)	46.6	71.7	72.0	78.1	83.3
	GPQA Diamond (Pass@1)	75.8	66.1	70.7	71.2	71.5
	SimpleQA (Correct)	30.3	17.8	28.2	24.9	30.1
	FRAMES (Acc.)	82.3	78.5	81.8	81.9	82.5
	AlpacaEval2.0 (LC-winrate)	24.7	50.1	55.8	62.1	87.6
	ArenaHard (GPT-4-1106)	53.6	77.0	73.2	75.6	92.3
Code	LiveCodeBench (Pass@1-COT)	50.0	57.5	63.5	64.6	65.9
	Codeforces (Percentile)	80.4	84.5	90.5	92.1	96.3
	Codeforces (Rating)	1444	1534	1687	1746	2029
	SWE Verified (Resolved)	43.2	39.6	44.6	45.6	49.2
	Aider-Polyglot (Acc.)	12.2	6.7	25.6	44.8	53.3
Math	AIME 2024 (Pass@1)	77.9	59.0	74.0	78.1	79.8
	MATH-500 (Pass@1)	95.9	94.2	95.9	95.4	97.3
	CNMO 2024 (Pass@1)	88.1	58.0	73.9	77.3	78.8
Chinese	CLUEWSC (EM)	93.1	92.8	92.6	91.6	92.8
	C-Eval (EM)	92.8	85.7	91.9	86.4	91.8
	C-SimpleQA (Correct)	66.4	58.8	64.2	66.9	63.7

- R1 Dev1
 - Instruction-following 능력 향상
 - cold-start 데이터 규모가 작아 추론 성능 일부 감소
- R1 Dev2
 - Reasoning 중심 RL 학습으로 추론 성능 향상
 - 사용자 선호 기반 벤치마크에서는 개선이 제한적
- R1 Dev3
 - Reasoning+Non-reasoning 데이터 SFT
 - 추론 능력 + 언어 생성 능력 동시 향상
- R1 Dev4

- 코드 , 수학 성능 소폭 향상
- Instruction-following 및 사용자 선호 성능 크게 개선

5. Conclusion

- 추론 능력이 고도화됨에 따라 Jailbreak 공격 시 모델이 생성하는 유해정보의 실행 가능성이 함께 높아짐
- DeepSeek-R1 모델은 GPT-4o와 유사한 정도의 안정성 확보
- Risk Control System과 결합 시 최고 수준 안정성 도달

⇒ 모델의 지능뿐만 아니라 이를 제어하는 안전 시스템과 조화가 필수

6. Limitations

- 정해진 형식의 출력 능력이 부족하고 외부 도구 활용 능력이 미흡
- 쉬운 문제에도 너무 많은 토큰을 소모하며 지나치게 깊게 생각하는 비효율성 존재
- 추론 과정 중 여전히 언어가 섞이는 현상 발생
- 중국의 데이터 검열 문제